Universidad Mariano Gálvez De Guatemala

Facultad de Ingeniería en Sistemas

Tarea II

Punteros y Memoria Dinámica

Josue David Flores Roldan Carnet: 9941-24-7857

Guatemala 21 de febrero de 2026

Parte I – Análisis de Memoria (0.5 puntos)

Dado el siguiente código:

```
#include <iostream>
using namespace std;

int main() {

   int x = 8;
   int* p = &x;
   int* q = p;

   *q = 20;

   cout << x << endl;
   cout << *p << endl;
   cout << *q << endl;

   return 0;
}
```

Responda:

1. **¿Cuál es la salida del programa?**

   R// 20

   20

   20

2. **¿Cuántos espacios de memoria distintos existen en el stack?**

   R// Hay tres espacios para memoria stack.

3. **¿Existe aliasing? Explique.**

   R// Si hay aliasing debido a que a que se crea la variable x y se el asigna un valor con esto después el puntero *p se le asigna la dirección de “x” después se declara *q como puntero también con la dirección a “x”. Se entiende que el aliasing es la situación en la que dos o mas variables hacen referencias a la misma ubicación de memoria.

Parte II – Memoria Dinámica (0.5 puntos)

Escriba un programa que:

1. **Reserve memoria dinámica para un entero.**

   R// int* p = new int;

2. **Asigne el valor 50 utilizando desreferenciación.**

   R// *p = 50;

3. **Imprima:**

   - **Dirección almacenada en el puntero.**

R// cout << "Direccion almacenada en el puntero: " << p << endl;

   - **Valor almacenado en esa dirección.**

R// cout << "Valor almacenado en esa direccion: " << *p << endl;

   - **Dirección del puntero.**

R// cout << "Direccion del puntero: " << &p << endl;

4. **Libere correctamente la memoria.**

   R// delete p;

5. **Asigne nullptr después del delete.**

   R// p = nullptr;

Programa completo

```
#include <iostream>
using namespace std;

int main() {

int* p = new int;
*p = 50;

cout << "Direccion almacenada en el puntero: " << p << endl;
```

```
cout << "Valor almacenado en esa direccion: " << *p << endl;
cout << "Direccion del puntero: " << &p << endl;

delete p;
p = nullptr;

return 0;
}
```

Parte III – Análisis de Error (0.5 puntos)

Analice el siguiente código:

```
int* p = new int(15);
int* q = p;

delete p;

cout << *q << endl;
```

Explique:

1. **¿Qué tipo de error conceptual existe?**

   R// Hay un erro de dangling pointer. Es debido al delete p debido a eso la memoria se libera, pero “q” todavía contiene la dirección de esa memoria.

2. **¿Por qué puede funcionar aparentemente?**

   R// Teóricamente funciona debido a que a pesar de que la memoria fue liberada, el contenido no se borra de manera instantánea. El sistema puede seguir funcionando aparentemente bien pero no es garantizado.

3. **¿Qué concepto del estándar de C++ se está violando?**

   R// El concepto que se viola es el de comportamiento indefinido debido a que si se accede a una memoria previamente borrada puede causar un comportamiento indefinido.

Parte IV – Memory Leak (0.5 puntos)

Dado el código:

```
int* p = new int(10);
p = new int(30);
```

Responda:

1. **¿Existe memory leak?**

   R// Si hay una fuga de memoria.

2. **¿Por qué ocurre?**

   R// Es debido a que al principio se crea para almacenar 10 valores pero después el puntero “p” se reasigna con 30 como no se ejecuta un delete esto hace que la primera diferencia se pierda.

3. **Reescriba el código para evitar el leak.**

   R// int* p = new int(10);
delete p;
p = new int(30);
delete p;
p = nullptr;
return 0;
}